Neural Networks and the Back-propagation Algorithm

Francisco S. Melo

In these notes, we provide a brief overview of the main concepts concerning neural networks and the back-propagation algorithm. We closely follow the presentation in [1]. We refer to [1, 2, 3] for further details.

Throughout these notes, random variables are represented with upper-case letters, such as X or Z. A sample of a random variable is represented by the corresponding lower-case letter, such as x or z. When random variables are vector valued, we use subscripts to indicate specific components, as in X_k or Z_k. The corresponding samples are represented using bold-face letters, such as $\mathbf{x}$ or $\mathbf{z}$, and individual components as x_k or z_k, respectively. When considering an indexed family of vector-valued data-points, we use indexed bold-face symbols to denote the elements in the family, as in $\mathbf{x}_n$ or $\mathbf{z}_n$.

1 The Perceptron

Artificial neural networks (ANNs) arose as an attempt to model mathematically the process by which information is handled by the brain. Learning methods based on neural networks are general and relatively simple to implement, making them a widely used class of methods when complex real-world data must be interpreted. Examples include recognition of handwritten digits, spoken words or faces.

ANNs correspond to networks of densely connected nodes, known as *neurons*, each of which is a small processing unit. The simplest model of such

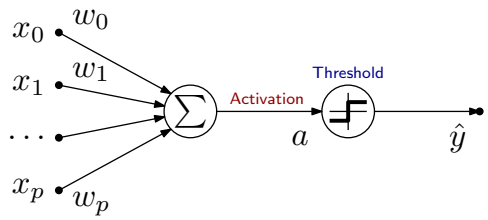

Figure 1: Representation of the Perceptron.

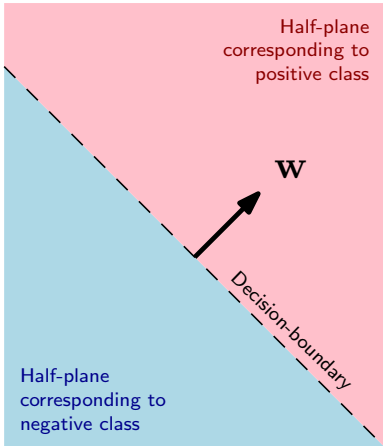

Figure 2: Decision boundary for the Perceptron, given the weight vector $\mathbf{w}$.

a network consists of a single unit, known as *perceptron*, and represented in the diagram of Fig. 1. The perceptron takes as input a vector $\mathbf{x} = [x_1, \ldots, x_p]$ of p real-valued inputs, from which it computes the *activation*, a, which is a linear combination of these inputs,

$$a = w_0 + \sum_{i=1}^{p} w_i x_i = \mathbf{w}^\top \mathbf{x}.$$

Note that we included one additional weight, w_0, that is independent of the input and is known as the *bias*. However, to provide a uniform treatment of the weights in the perceptron, it is customary to consider one additional input, x_0, that is constant and equal to 1, i.e., $x_0 \equiv 1$. We included this "fictitious" input in the representation of Fig. 1.

The output of the perceptron, $\hat{y}$, is computed as the image of the activation a by a *threshold function* σ,

$$\hat{y}(\mathbf{x}) \triangleq \sigma(a) = \begin{cases} 1 & \text{if } a > 0 \\ \text{-1} & \text{otherwise.} \end{cases}$$

The perceptron can then be used for binary classification tasks, where the inputs $\mathbf{x}$ for which $\hat{y}(\mathbf{x}) = 1$ correspond to the positive instances and those for which $\hat{y}(\mathbf{x}) = -1$ correspond to the negative instances. Geometrically, the data-points classified by the perceptron as belonging to the positive class correspond to those data-points $\mathbf{x}$ whose inner product with the weight vector $\mathbf{w}$ is positive (see Fig. 2).

1.1 Perceptron Learning Rule

To determine the process by which the perceptron is trained, it is necessary to define an error function with respect to which the performance of the

perceptron is to be measured (remember that this is one of the fundamental elements necessary to define a learning task). While the number of misclassified data-points is a natural candidate for error function, it is not amenable to an easy analytical treatment.

Instead, we introduce the so-called *perceptron criterion*. Note, first of all, that a data-point $\mathbf{x}$ in class y (with $y \in \{-1, 1\}$) is properly classified by the perceptron if $\mathbf{w}^\top \mathbf{x} y > 0$. Given a training data-set $\mathcal{D}$, let $\mathcal{M}$ denote the set of *misclassified* data-points. The perceptron criterion tries to minimize the error

$$\mathcal{E}(\mathbf{w}) = -\sum_{\mathbf{x} \in \mathcal{M}} \mathbf{w}^\top \mathbf{x}_n y_n.$$

To minimize the error $\mathcal{E}$, we adopt a general *gradient descent approach*, whereby the minimum of a general real-valued function $F(\mathbf{z})$ is gradually approximated by the sequence $\left\{\mathbf{z}^{(1)}, \mathbf{z}^{(2)}, \ldots\right\}$ defined recursively by

$$\mathbf{z}^{(\tau+1)} = \mathbf{z}^{(\tau)} - \eta \nabla_{\mathbf{z}} F(\mathbf{z}^{(\tau)}),$$

where η is a positive step-size. Specifically in the case of the perceptron, the weight vector $\mathbf{w}$ is adjusted as

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{E}(\mathbf{w}) = \mathbf{w} + \eta \sum_{n \in \mathcal{M}} \mathbf{x}_n y_n. \tag{1}$$

Interestingly, two modifications are generally considered to the learning rule in (1). The first is to consider incremental updates, where the weight vector is updated one data-point at a time. The second modification arises from noting that the output of the perceptron remains unchanged if $\mathbf{w}$ is multiplied by a constant, which allows us to consider a step-size $\eta = 1$.

The training process for the perceptron can thus be summarized as follows. Given the data-set $\mathcal{D} = \{(\mathbf{x}_n, y_n), n = 1, \ldots, N\}$,

1. For each pair $(\mathbf{x}_n, y_n) \in \mathcal{D}$, if $\mathbf{w}^\top \mathbf{x}_n y_n > 0$, move to the next pair.

2. Otherwise, adjust $\mathbf{w}$ according to the learning rule:

$$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}_n y_n. \tag{2}$$

While the training rule for perceptrons is straightforward to implement, perceptrons are restricted to *linear decision boundaries*, which means that they are unable to learn classifiers for data that is not linearly separable.

2 Multilayer Perceptron

A *multilayer perceptron* (MLP), also known as feed-forward neural network or multilayer feedforward network, is a network of densely connected units

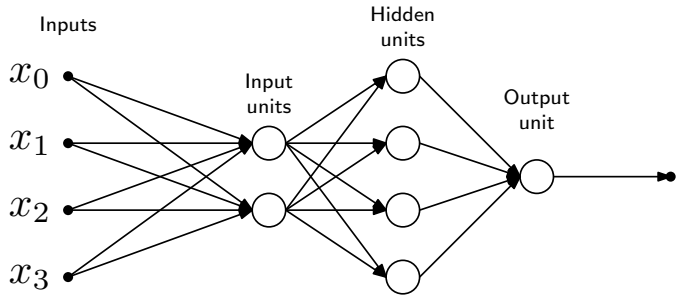

Figure 3: Example of a multilayer perceptron with two input units, four hidden units and one output unit.

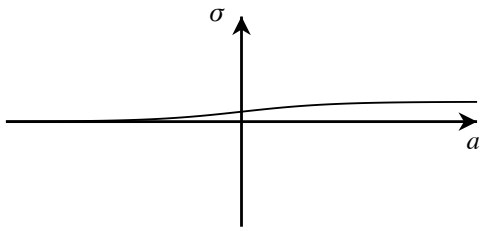

Figure 4: Sigmoid threshold function $\sigma(a) = \frac{1}{1+\exp(-a)}$.

similar to the perceptron discussed in Section 1 and having a non-linear threshold function. The units in a MLP are arranged in layers—units connected directly to the inputs of the network constitute the *input layer* of the network, while those whose output corresponds to the output of the network constitute the *output layer* of the network. All other intermediate layers are referred as *hidden layers*. An example of a multilayer perceptron is depicted in Fig. 3.

Multilayer perceptrons are able to represent a much richer set of functions than those representable using a single perceptron. In fact, the *universal approximation theorem* states that a multilayer perceptron with a single hidden layer that contains finite number of hidden neurons and with an arbitrary activation function can approximate with an arbitrarily small error any continuous function defined over any compact subset of $\mathbb{R}^p$.

Each unit in a MLP is similar to the perceptron surveyed in Section 1 and depicted in Fig. 1. However, for purposes of training, it is convenient that the output(s) of the network are differentiable functions of the inputs, for which reason the neurons in a MLP usually are defined with differential threshold functions. A common threshold function is the *logistic sigmoid function*,

$$\sigma(a) = \frac{1}{1 + \exp(-a)},$$

depicted in Fig. 4.

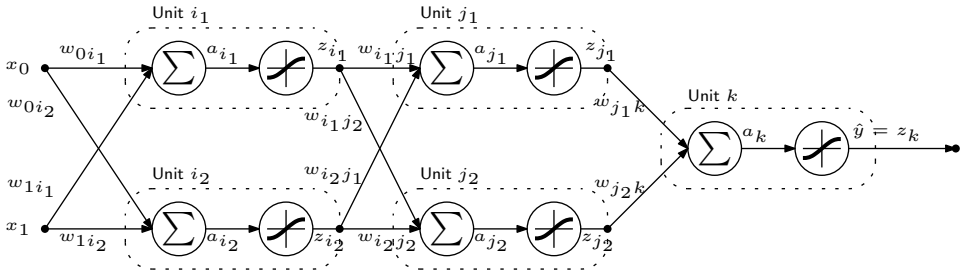

Figure 5: Artificial neural network with 1 hidden layer.

The output of the network can be computed by propagating the input information throughout the network, in a process known as *forward propagation*. To illustrate this process, consider the ANN model depicted in detail in Fig. 5. We denote by $\mathbf{w}_i$ the weights associated with unit i and by w_{ij} the weight associated with the connection between the output of unit i and unit j.

Given the input vector $\mathbf{x}$ to the network, the output of input units i_1 and i_2 is given by

$$z_{i_1} = \sigma(\mathbf{w}_{i_1}^\top \mathbf{x}) \qquad\qquad z_{i_2} = \sigma(\mathbf{w}_{i_2}^\top \mathbf{x}),$$

which corresponds to the forward propagation of the input $\mathbf{x}$ through the first layer in the network. The two outputs z_{i_1} and z_{i_2} now act as inputs for the second layer in the network. Letting $\mathbf{z}_i = [z_{i_1}, z_{i_2}]^\top$, it follows that

$$z_{j_1} = \sigma(\mathbf{w}_{j_1}^\top \mathbf{z}_i) \qquad\qquad z_{j_2} = \sigma(\mathbf{w}_{j_2}^\top \mathbf{z}_i),$$

which corresponds to the propagation of $\mathbf{z}_i$ through the second layer in the network. Finally, the output of the network is given by

$$\hat{y}(\mathbf{x}) = z_k = \sigma(\mathbf{w}_k^\top \mathbf{z}_j)$$

where, as before, we defined $\mathbf{z}_j = [z_{j_1}, z_{j_2}]^\top$.

2.1 The Back-propagation Algorithm

As before, to determine the process by which a MLP can be trained, it is necessary to define an error function with respect to which the performance of the MLP is to be measured. Given a data-set $\mathcal{D} = \{(\mathbf{x}_n, y_n), n = 1, \ldots, N\}$, we adopt the error function

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} \big(\hat{y}(\mathbf{x}_n) - y_n\big)^2.$$

We have, for simplicity, considered the case where there is one single output $\hat{y}$ to the network, but the reasoning can be trivially replicated to accommodate vector outputs.

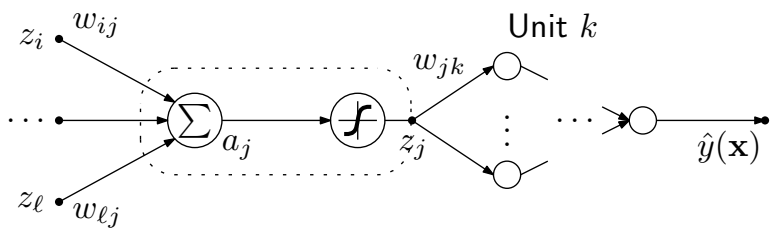

Figure 6: Unit j in the network.

To minimize the error $\mathcal{E}$, we again adopt a gradient descent approach, where weights in the networ, $\mathbf{w}$, are adjusted according to the rule

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{E}(\mathbf{w}),$$

where $\nabla_{\mathbf{w}} \mathcal{E}(\mathbf{w})$ denotes the gradient of the error function with respect to the weights in the network. The back-propagation algorithm allows for a simple and efficient way of propagating the error information backwards in the network, allowing for successive updates of the weights from the output to the input. To derive the back-propagation learning rule, we start by writing the error function as

$$\mathcal{E}(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}),$$

with

$$E_n(\mathbf{w}) = \frac{1}{2}\big(\hat{y}(\mathbf{x}_n) - y_n\big)^2.$$

As with the perceptron, the updates to the weights can be done incrementally, using one data-point at a time, using instead the update rule

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} E_n(\mathbf{w}).$$

It remains to determine the gradient $\nabla_{\mathbf{w}} E(\mathbf{w})$. Let us then focus on one particular unit in the network—say, unit j—and determine the components of $\nabla_{\mathbf{w}} E_n(\mathbf{w})$ comprising the derivatives of $E_n(\mathbf{w})$ with respect the weights in unit j, w_{ij} (see Fig. 6).

We start by writing the derivative with respect to w_{ij} as

$$\frac{\partial E_n}{\partial w_{ij}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ij}}.$$

To simplify the notation, we henceforth write

$$\delta_j = \frac{\partial E_n}{\partial a_j}.$$

Moreover, it follows from the definition of activation that

$$\frac{\partial a_j}{\partial w_{ij}} = z_i.$$

Combining the two, we get

$$\frac{\partial E_n}{\partial w_{ij}} = \delta_j z_i. \tag{3}$$

Let us now compute the term δ_j. If j is the output unit, then

$$E_n = \frac{1}{2}(\sigma(a_j) - y_n)^2,$$

and we immediately get

$$\delta_j = \sigma'(a_j)(\sigma(a_j) - y_n) \tag{4}$$

which, in the case of the logistic sigmoid function, yields

$$\delta_j = \sigma(a_j)(1 - \sigma(a_j))(\sigma(a_j) - y_n).$$

On the other hand, if j is not an output function, E_n depends on a_j through all units k to which unit j is connected (see Fig. 6). In other words,

$$\frac{\partial E_n}{\partial a_j} = \sum_{k=1}^{K} \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j} = \sum_{k=1}^{K} \delta_k \frac{\partial a_k}{\partial a_j}.$$

Finally, we have that

$$\frac{\partial a_k}{\partial a_j} = w_{jk}\sigma'(a_j),$$

and thus

$$\delta_j = \sigma'(a_j) \sum_{k=1}^{K} w_{jk}\delta_k. \tag{5}$$

Note that, as evidenced in (5), the derivative δ_j for unit j can be computed by propagating the derivatives δ_k of the *subsequent nodes* through the network. In conclusion, the back-propagation algorithm can be summarized as follows. Given the data-set $\mathcal{D} = \{(\mathbf{x}_n, y_n), n = 1, \ldots, N\}$,

1. For each pair $(\mathbf{x}_n, y_n) \in \mathcal{D}$ forward propagate the input $\mathbf{x}_n$ through the network to compute $\hat{y}(\mathbf{x}_n)$. In this process, compute the activations a_j for all hidden and output units.

2. Evaluate δ_j for the output units using (4).

3. Back-propagate the δ's using (5), determining δ_j for all hidden units in the network.

4. For all nodes in the network, compute the derivatives $\frac{\partial E_n}{\partial w_{ij}}$ using (3).

5. Update each weight w_{ij} using the rule

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial E_n}{\partial w_{ij}}. \tag{6}$$

References

[1] Christopher M. Bishop. *Pattern Recognition and Machine Learning.* Springer Science, 2006.

[2] Simon Haykin. *Neural Networks: A Comprehensive Foundation.* Prentice Hall, 2nd edition, 1998.

[3] Tom M. Mitchell. *Machine Learnin.* McGraw-Hill, 1997.